05

Escrevendo SPECs que a IA cumpre

"Qual o percentual de aderência do Claude Code a uma SPEC?" — pergunta errada. Não existe percentual fixo, porque aderência não é uma propriedade do modelo: é uma propriedade da *sua* SPEC. Este material mostra o que aumenta e o que destrói a aderência, com estimativas realistas por cenário e um checklist para avaliar a sua especificação antes de entregá-la a um agent.

A IDEIA CENTRAL DESTE MATERIAL

O agent não executa a SPEC — ele a interpreta. Toda ambiguidade vira uma decisão autônoma do modelo, que pode ou não ser a que você queria. Quem controla a aderência é quem escreve.

1 O que aumenta a aderência

FATOR	IMPACTO	POR QUÊ
SPEC clara, sem ambiguidade	**Alto**	O modelo não precisa "adivinhar" — menos interpretação, menos erro.
Exemplos concretos e casos de borda	**Alto**	Ancoram a interpretação em cenários reais, não em abstrações.
Tarefas menores e bem delimitadas	**Alto**	Menos contexto para gerenciar, menos chance de divergir no caminho.
Revisão humana por checkpoint	**Alto**	Corrigir depois de 1 seção é barato; depois de 10, é caro.
TDD — testes guiando a implementação	**Alto**	Teste é especificação executável: se passa, está aderente.
CLAUDE.md com regras do projeto	Médio	Dá o contexto global (convenções, padrões) — mas não substitui a SPEC específica.

Ambiguidade na prática

✗ SPEC AMBÍGUA

"Criar um sistema de login seguro."

O modelo decide sozinho: session ou token? Expiração de 5min ou 1h? Email obrigatório?

✓ SPEC CLARA

"Autenticação JWT com refresh token, access token expira em 15min, validação de email obrigatória, rate limiting de 5 tentativas/min em /auth/login."

Exemplos são testes de especificação

✗ SEM EXEMPLO

"A API deve paginar resultados."

✓ COM EXEMPLO

`GET /api/items?page=2&limit=20` retorna `{ data: [...], total: 150, page: 2, totalPages: 8 }`. Se page excede `totalPages`: array vazio, mesmos campos.

O segundo formato elimina dúvidas sobre formato de resposta, nome de campos e comportamento nos casos-limite — de uma vez.

2 O que destrói a aderência

FATOR	IMPACTO	POR QUÊ
Requisitos implícitos	**Alto**	O que não está escrito, o modelo infere. Inferência = chance de erro.
Requisitos contraditórios ou vagos	**Alto**	Contradição força o modelo a escolher um lado — pode não ser o seu.
Tarefa gigante num único prompt	**Alto**	O contexto se dilui; decisões do início se perdem no caminho.
Sem revisão intermediária	**Alto**	Erros do começo propagam e amplificam até o fim.
Dependências externas não documentadas	Médio	O modelo não sabe que a API tem rate limit ou que o serviço está em manutenção.

O perigo dos requisitos implícitos: são aqueles que você "sabe", mas não escreveu. Para um dev experiente no projeto, são óbvios. Para um modelo que nunca viu o projeto, são **invisíveis**. "O sistema deve funcionar" não diz nada sobre error handling, retry, fallback ou timeout. A versão explícita diz: "todo endpoint retorna 500 com mensagem genérica em exception não tratada; integrações externas têm retry com backoff exponencial (3 tentativas: 1s, 2s, 4s); timeout de 10s em chamadas HTTP".

3 Estimativa realista por cenário

CENÁRIO	CARACTERÍSTICAS	ADERÊNCIA ESPERADA
1 — Simples, clara, pequena	1–2 módulos, exemplos concretos por funcionalidade, critérios testáveis, tarefa < 1h	**~85–95%** — funciona quase de primeira; ajustes de detalhe.
2 — Média, com implícitos	3–4 módulos, exemplos parciais, critérios nem todos testáveis, alguns "óbvios" não escritos	**~60–75%** — estrutura correta, detalhes divergem; 1–2 rodadas de correção.
3 — Grande, sem checkpoints	5+ módulos interdependentes, texto narrativo sem exemplos, sem fases, sem critérios testáveis	**~30–50%** — começa bem, diverge no meio, retrabalho significativo.
4 — Ambígua ou contraditória	Requisitos que se contradizem, linguagem vaga ("deve ser bom"), mistura problema com solução	**Imprevisível (pode ser 0%)** — o pior caso: parece certo por fora, com erros profundos por dentro.

> Esses percentuais não são promessa de ferramenta — são consequência direta dos fatores da seção 1 e 2. Mover sua SPEC do cenário 3 para o cenário 1 está **100% sob seu controle**.

4 A estratégia que funciona na prática

1. **SPEC em seções pequenas e verificáveis.** Não escreva tudo de uma vez. Cada seção deve poder ser implementada e verificada de forma independente.
2. **Checkpoint após cada seção.** O ciclo não é "escrever SPEC → executar tudo → revisar no final". É "executar seção → revisar (`git diff`) → ajustar a SPEC se necessário → próxima seção".
3. **Teste como critério de aceitação, não como opcional.** Se a SPEC diz "deve fazer X", existe um teste que verifica X. O teste é a prova objetiva de aderência.
4. **Iterar, não confiar no "gerou certo de primeira".** Mesmo com SPEC perfeita, revise. O agent não é um contratado que entrega 100% no prazo — é um colaborador que precisa de feedback contínuo.

5 Checklist: avalie sua SPEC antes de entregar

- ☐ Cada funcionalidade tem pelo menos **1 exemplo concreto**?
- ☐ Não existem **requisitos implícitos** — tudo que importa está escrito?
- ☐ A SPEC está dividida em **seções executáveis independentemente**?
- ☐ Existem **critérios de aceitação testáveis** para cada funcionalidade?
- ☐ Os **casos de borda** estão documentados (o que acontece quando dá erro)?
- ☐ **Stack e dependências** estão explícitas?

- ☐ O que está **fora do escopo** está claramente delimitado?

> Sete "sim" = sua SPEC está no cenário 1 (~85–95%). Qualquer "não" aponta exatamente onde ela vai falhar.

▸ COLA PARA FIXAR

- Aderência é propriedade da **SPEC**, não do modelo: toda ambiguidade vira decisão autônoma do agent.
- **Exemplos concretos** valem mais que descrições: são testes de especificação.
- Requisito **implícito** é requisito invisível — o que você não escreveu, o modelo inventa.
- Seções pequenas + **checkpoint** após cada uma: corrigir cedo é barato, corrigir tarde é caro.
- Se a SPEC diz "deve fazer X", existe um **teste** que verifica X.
- SPEC clara e pequena chega a **~85–95%**; ambígua e gigante pode entregar zero.